

In []:

```
In [71]: import pandas_ta as pta
import numpy as np
import pandas as pd
import mplfinance as mpf
from tabulate import tabulate
from utils.KrakenHistoricalData import KrakenHistoricalData
from ggTrader.Signals import Signals
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Rectangle
import matplotlib.dates as mdates
```

```
In [72]: # Ticker
symbols = ["BTC"]
interval = "4h"

# Time Range

end = pd.to_datetime("2025-06-30").tz_localize('UTC')
start = end - pd.Timedelta(days=30 * 6)

# Indicator Params
sar_acceleration = 0.02
sar_maximum = 0.2
atr_multiplier = 3
adx_threshold = 25
ce_high_length = 22
#
k = KrakenHistoricalData()
s = Signals()

df_multi = k.get_ohlcv_df(symbols, interval=interval)
print(df_multi.head())

# remove multi-index
df_multi.columns = df_multi.columns.droplevel(0)
df = df_multi.loc[start:end]

print(f"\nMultiIndex Removed")
print(df.head())
```

		BTC		
		open	high	low
2023-01-01 00:00:00+00:00		16528.699219	16530.000000	16505.199219
2023-01-01 04:00:00+00:00		16519.300781	16542.400391	16506.900391
2023-01-01 08:00:00+00:00		16512.400391	16538.400391	16490.000000
2023-01-01 12:00:00+00:00		16500.199219	16550.000000	16497.699219
2023-01-01 16:00:00+00:00		16550.000000	16573.099609	16531.199219

		close	volume	trades	base	quote
2023-01-01 00:00:00+00:00		16519.300781	2.636628e+10	803	BTC	USD
2023-01-01 04:00:00+00:00		16510.699219	1.122105e+11	2919	BTC	USD
2023-01-01 08:00:00+00:00		16500.199219	1.014384e+11	2536	BTC	USD
2023-01-01 12:00:00+00:00		16549.900391	2.684441e+10	2002	BTC	USD
2023-01-01 16:00:00+00:00		16553.699219	3.952236e+10	2406	BTC	USD

MultiIndex Removed

		open	high	low	
2025-01-01 00:00:00+00:00		93370.796875	94250.000000	93282.000000	
2025-01-01 04:00:00+00:00		93625.101562	93654.500000	93327.796875	
2025-01-01 08:00:00+00:00		93518.703125	93535.703125	92588.101562	
2025-01-01 12:00:00+00:00		93278.101562	94200.000000	93278.000000	
2025-01-01 16:00:00+00:00		94189.796875	94581.203125	93603.796875	

		close	volume	trades	base	quote
2025-01-01 00:00:00+00:00		93625.101562	9.523742e+11	6035	BTC	USD
2025-01-01 04:00:00+00:00		93518.703125	4.491399e+11	2874	BTC	USD
2025-01-01 08:00:00+00:00		93278.101562	7.846995e+11	2963	BTC	USD
2025-01-01 12:00:00+00:00		94189.796875	1.138570e+12	4533	BTC	USD
2025-01-01 16:00:00+00:00		94440.101562	1.040107e+12	4673	BTC	USD

```
In [73]: # Signals
signals = df.copy()
# moving average
# ma = pta.ohlcv4(df['open'], df['high'], df['low'], df['close'])
ma = pta.zlma(df['close'], 10)

# Calculate technical indicators
signals['rsi'] = pta.rsi(df['close'])

# adx > 25 strong trend
adx = pta.adx(df['high'], df['low'], df['close'])

# Entry: SAR Signal
psar = pta.psar(df['high'],
                df['low'],
                close=df['close'])
print(f"\nSAR:")

signals['sar'] = psar.iloc[:, 0]
signals['sar_s'] = np.where(signals['sar'] > 0, True, False)

print(signals[['sar', 'sar_s']].tail()) # Entry Signals

print(f"\nADX:")
print(tabulate(adx.tail(), headers="keys", tablefmt="github"))
```

```

signals['adx'] = adx.iloc[:, 1]
signals['adx_s'] = np.where(signals['adx'] > adx_threshold, True, False)

# Exit: Chandelier Exit uses ATR
signals['atr'] = pta.atr(df['high'], df['low'], df['close'])
ce = pta.chandelier_exit(df['high'],
                        df['low'],
                        df['close'],
                        multiplier=atr_multiplier, high_length=ce_high_length)

print(f"\nChandelier_Raw")
print(tabulate(ce.tail(), headers="keys", tablefmt="github"))
signals['ce_l'] = np.where(ce.iloc[:, 2] > 0, ce.iloc[:, 0], np.nan)
signals['ce_sh'] = np.where(ce.iloc[:, 2] < 0, ce.iloc[:, 1], np.nan)
signals['ce_exit'] = np.where(ce.iloc[:, 2] == 1, False, True)
print(f"\nChandelier Exit:")
print(signals[['ce_l', 'ce_sh', 'ce_exit']].tail(50))

# entry
print(f"\nEntry Signals:")
# Have ADX strength, and sar is a buy and ce is not an exit
entry_series = signals['adx_s'] & signals['sar_s'] & ~signals['ce_exit']
entry_rise = entry_series & (~entry_series.shift(1, fill_value=False))
print(entry_rise.tail(50))

print(f"\nExit Signals:")
exit_series = signals['ce_exit']
print(exit_series.tail(20))
exit_rise = exit_series & (~exit_series.shift(1, fill_value=False))
print(exit_rise.tail(20))

```

SAR:

		sar	sar_s
2025-06-29 08:00:00+00:00	106477.203125	True	
2025-06-29 12:00:00+00:00	106516.707031	True	
2025-06-29 16:00:00+00:00	106594.466875	True	
2025-06-29 20:00:00+00:00	106669.116325	True	
2025-06-30 00:00:00+00:00	106740.779797	True	

ADX:

	ADX_14	ADXR_14_2	DMP_14	DMN_14
2025-06-29 08:00:00+00:00	16.5153	15.9498	2608.55	1239.24
2025-06-29 12:00:00+00:00	17.8882	16.4682	2430.53	1150.73
2025-06-29 16:00:00+00:00	18.3559	17.4356	2256.92	1370.53
2025-06-29 20:00:00+00:00	19.6316	18.7599	2717.82	1272.64
2025-06-30 00:00:00+00:00	21.2165	19.7862	2880.58	1181.73

Chandler_Raw

	CHDLREXTl_22_22_14_3	CHDLREXTs_22_22_14_3	CHDLREXTd_22_22_14_3
2025-06-29 08:00:00+00:00	106364	108565	
2025-06-29 12:00:00+00:00	106357	108581	
2025-06-29 16:00:00+00:00	106417	108521	
2025-06-29 20:00:00+00:00	106322	108616	
2025-06-30 00:00:00+00:00	106701	108568	

Chandelier Exit:

	ce_l	ce_sh	ce_exit
2025-06-21 20:00:00+00:00	NaN	104022.039583	True
2025-06-22 00:00:00+00:00	NaN	104122.536755	True
2025-06-22 04:00:00+00:00	NaN	104034.204889	True
2025-06-22 08:00:00+00:00	NaN	103946.374964	True
2025-06-22 12:00:00+00:00	NaN	102572.749408	True
2025-06-22 16:00:00+00:00	NaN	102412.272330	True
2025-06-22 20:00:00+00:00	NaN	102307.983905	True
2025-06-23 00:00:00+00:00	NaN	102253.920322	True
2025-06-23 04:00:00+00:00	NaN	102195.125232	True
2025-06-23 08:00:00+00:00	NaN	102099.237939	True
2025-06-23 12:00:00+00:00	NaN	102191.677974	True
2025-06-23 16:00:00+00:00	102059.545117	NaN	False
2025-06-23 20:00:00+00:00	101728.099372	NaN	False
2025-06-24 00:00:00+00:00	101539.751315	NaN	False
2025-06-24 04:00:00+00:00	101558.438163	NaN	False
2025-06-24 08:00:00+00:00	101693.070370	NaN	False
2025-06-24 12:00:00+00:00	101816.673044	NaN	False
2025-06-24 16:00:00+00:00	102181.999412	NaN	False
2025-06-24 20:00:00+00:00	102352.121664	NaN	False
2025-06-25 00:00:00+00:00	102930.276478	NaN	False
2025-06-25 04:00:00+00:00	103062.342109	NaN	False

2025-06-25 08:00:00+00:00	103709.924815	NaN	False
2025-06-25 12:00:00+00:00	104457.122708	NaN	False
2025-06-25 16:00:00+00:00	104517.770751	NaN	False
2025-06-25 20:00:00+00:00	104599.886456	NaN	False
2025-06-26 00:00:00+00:00	104749.430281	NaN	False
2025-06-26 04:00:00+00:00	104899.764502	NaN	False
2025-06-26 08:00:00+00:00	104930.267372	NaN	False
2025-06-26 12:00:00+00:00	104938.833654	NaN	False
2025-06-26 16:00:00+00:00	105050.266295	NaN	False
2025-06-26 20:00:00+00:00	105042.654082	NaN	False
2025-06-27 00:00:00+00:00	105043.107361	NaN	False
2025-06-27 04:00:00+00:00	105106.570117	NaN	False
2025-06-27 08:00:00+00:00	105237.628725	NaN	False
2025-06-27 12:00:00+00:00	105246.270200	NaN	False
2025-06-27 16:00:00+00:00	105207.687284	NaN	False
2025-06-27 20:00:00+00:00	105344.701808	NaN	False
2025-06-28 00:00:00+00:00	105462.330250	NaN	False
2025-06-28 04:00:00+00:00	105616.727420	NaN	False
2025-06-28 08:00:00+00:00	105767.040417	NaN	False
2025-06-28 12:00:00+00:00	105858.680387	NaN	False
2025-06-28 16:00:00+00:00	105934.903551	NaN	False
2025-06-28 20:00:00+00:00	106069.045485	NaN	False
2025-06-29 00:00:00+00:00	106169.371477	NaN	False
2025-06-29 04:00:00+00:00	106289.616707	NaN	False
2025-06-29 08:00:00+00:00	106364.185379	NaN	False
2025-06-29 12:00:00+00:00	106356.926044	NaN	False
2025-06-29 16:00:00+00:00	106416.552979	NaN	False
2025-06-29 20:00:00+00:00	106321.791940	NaN	False
2025-06-30 00:00:00+00:00	106701.296757	NaN	False

Entry Signals:

2025-06-21 20:00:00+00:00	False
2025-06-22 00:00:00+00:00	False
2025-06-22 04:00:00+00:00	False
2025-06-22 08:00:00+00:00	False
2025-06-22 12:00:00+00:00	False
2025-06-22 16:00:00+00:00	False
2025-06-22 20:00:00+00:00	False
2025-06-23 00:00:00+00:00	False
2025-06-23 04:00:00+00:00	False
2025-06-23 08:00:00+00:00	False
2025-06-23 12:00:00+00:00	False
2025-06-23 16:00:00+00:00	True
2025-06-23 20:00:00+00:00	False
2025-06-24 00:00:00+00:00	False
2025-06-24 04:00:00+00:00	False
2025-06-24 08:00:00+00:00	False
2025-06-24 12:00:00+00:00	False
2025-06-24 16:00:00+00:00	False
2025-06-24 20:00:00+00:00	False
2025-06-25 00:00:00+00:00	False
2025-06-25 04:00:00+00:00	False
2025-06-25 08:00:00+00:00	False
2025-06-25 12:00:00+00:00	False
2025-06-25 16:00:00+00:00	False
2025-06-25 20:00:00+00:00	False

2025-06-26 00:00:00+00:00	False
2025-06-26 04:00:00+00:00	True
2025-06-26 08:00:00+00:00	False
2025-06-26 12:00:00+00:00	False
2025-06-26 16:00:00+00:00	False
2025-06-26 20:00:00+00:00	False
2025-06-27 00:00:00+00:00	False
2025-06-27 04:00:00+00:00	False
2025-06-27 08:00:00+00:00	False
2025-06-27 12:00:00+00:00	False
2025-06-27 16:00:00+00:00	False
2025-06-27 20:00:00+00:00	False
2025-06-28 00:00:00+00:00	False
2025-06-28 04:00:00+00:00	False
2025-06-28 08:00:00+00:00	False
2025-06-28 12:00:00+00:00	False
2025-06-28 16:00:00+00:00	False
2025-06-28 20:00:00+00:00	False
2025-06-29 00:00:00+00:00	False
2025-06-29 04:00:00+00:00	False
2025-06-29 08:00:00+00:00	False
2025-06-29 12:00:00+00:00	False
2025-06-29 16:00:00+00:00	False
2025-06-29 20:00:00+00:00	False
2025-06-30 00:00:00+00:00	False

Freq: 4h, dtype: bool

Exit Signals:

2025-06-26 20:00:00+00:00	False
2025-06-27 00:00:00+00:00	False
2025-06-27 04:00:00+00:00	False
2025-06-27 08:00:00+00:00	False
2025-06-27 12:00:00+00:00	False
2025-06-27 16:00:00+00:00	False
2025-06-27 20:00:00+00:00	False
2025-06-28 00:00:00+00:00	False
2025-06-28 04:00:00+00:00	False
2025-06-28 08:00:00+00:00	False
2025-06-28 12:00:00+00:00	False
2025-06-28 16:00:00+00:00	False
2025-06-28 20:00:00+00:00	False
2025-06-29 00:00:00+00:00	False
2025-06-29 04:00:00+00:00	False
2025-06-29 08:00:00+00:00	False
2025-06-29 12:00:00+00:00	False
2025-06-29 16:00:00+00:00	False
2025-06-29 20:00:00+00:00	False
2025-06-30 00:00:00+00:00	False

Freq: 4h, Name: ce_exit, dtype: bool

2025-06-26 20:00:00+00:00	False
2025-06-27 00:00:00+00:00	False
2025-06-27 04:00:00+00:00	False
2025-06-27 08:00:00+00:00	False
2025-06-27 12:00:00+00:00	False
2025-06-27 16:00:00+00:00	False
2025-06-27 20:00:00+00:00	False

```

2025-06-28 00:00:00+00:00    False
2025-06-28 04:00:00+00:00    False
2025-06-28 08:00:00+00:00    False
2025-06-28 12:00:00+00:00    False
2025-06-28 16:00:00+00:00    False
2025-06-28 20:00:00+00:00    False
2025-06-29 00:00:00+00:00    False
2025-06-29 04:00:00+00:00    False
2025-06-29 08:00:00+00:00    False
2025-06-29 12:00:00+00:00    False
2025-06-29 16:00:00+00:00    False
2025-06-29 20:00:00+00:00    False
2025-06-30 00:00:00+00:00    False
Freq: 4h, Name: ce_exit, dtype: bool

```

In [74]: *# filter entries and exits, one for one entry/exit*

```

in_pos = False
filtered_entry = pd.Series(False, index=signals.index)
filtered_exit = pd.Series(False, index=signals.index)

for ts in signals.index:
    if not in_pos and entry_rise.loc[ts]:
        filtered_entry.loc[ts] = True
        in_pos = True
    elif in_pos and exit_rise.loc[ts]:
        filtered_exit.loc[ts] = True
        in_pos = False
    # elif in_pos and exit_series.loc[ts]:
    #     filtered_exit.loc[ts] = True
    #     in_pos = False
# filtered_exit = filtered_entry.shift(-1, fill_value=False)
i_start = 40 * 6
i_end = 50 * 6
entry_concat = pd.concat([entry_series.iloc[i_start:i_end], filtered_entry.iloc[i_s
exit_concat = pd.concat([exit_series.iloc[i_start:i_end], filtered_exit.iloc[i_star

print(f"\nFiltered Entry/Exit Signals:")
print(pd.concat([entry_concat, exit_concat, signals['ce_l']], axis=1).iloc[i_start:

print(f"\nChandelier Exit:")
print(signals[['ce_l', 'ce_sh', 'ce_exit']].iloc[i_start:i_end])

print(f"\nExit Prices:")
print(signals['ce_l'].loc[filtered_exit.shift(-1, fill_value=False).values])

```

Filtered Entry/Exit Signals:

	0	1	ce_exit	0	ce_l
2025-02-10 00:00:00+00:00	False	False	True	False	NaN
2025-02-10 04:00:00+00:00	False	False	True	False	NaN
2025-02-10 08:00:00+00:00	False	False	True	False	NaN
2025-02-10 12:00:00+00:00	False	False	True	False	NaN
2025-02-10 16:00:00+00:00	False	False	True	False	NaN
2025-02-10 20:00:00+00:00	False	False	True	False	NaN
2025-02-11 00:00:00+00:00	False	False	True	False	NaN
2025-02-11 04:00:00+00:00	False	False	False	False	95450.588651
2025-02-11 08:00:00+00:00	False	False	False	False	95477.928859
2025-02-11 12:00:00+00:00	False	False	False	False	95368.990860
2025-02-11 16:00:00+00:00	False	False	True	False	NaN
2025-02-11 20:00:00+00:00	False	False	True	False	NaN
2025-02-12 00:00:00+00:00	False	False	True	False	NaN
2025-02-12 04:00:00+00:00	False	False	True	False	NaN
2025-02-12 08:00:00+00:00	False	False	True	False	NaN
2025-02-12 12:00:00+00:00	False	False	True	False	NaN
2025-02-12 16:00:00+00:00	False	False	True	False	NaN
2025-02-12 20:00:00+00:00	False	False	True	False	NaN
2025-02-13 00:00:00+00:00	False	False	True	False	NaN
2025-02-13 04:00:00+00:00	False	False	True	False	NaN
2025-02-13 08:00:00+00:00	False	False	True	False	NaN
2025-02-13 12:00:00+00:00	False	False	True	False	NaN
2025-02-13 16:00:00+00:00	False	False	True	False	NaN
2025-02-13 20:00:00+00:00	False	False	True	False	NaN
2025-02-14 00:00:00+00:00	False	False	True	False	NaN
2025-02-14 04:00:00+00:00	False	False	True	False	NaN
2025-02-14 08:00:00+00:00	False	False	True	False	NaN
2025-02-14 12:00:00+00:00	False	False	True	False	NaN
2025-02-14 16:00:00+00:00	False	False	False	False	95352.442621
2025-02-14 20:00:00+00:00	False	False	False	False	95286.196720
2025-02-15 00:00:00+00:00	False	False	False	False	95405.603427
2025-02-15 04:00:00+00:00	False	False	False	False	95560.752847
2025-02-15 08:00:00+00:00	False	False	False	False	95706.320166
2025-02-15 12:00:00+00:00	False	False	False	False	95843.868726
2025-02-15 16:00:00+00:00	False	False	False	False	95982.971295
2025-02-15 20:00:00+00:00	False	False	False	False	96130.780823
2025-02-16 00:00:00+00:00	False	False	False	False	96275.296478
2025-02-16 04:00:00+00:00	False	False	False	False	96368.003538
2025-02-16 08:00:00+00:00	False	False	False	False	96486.231522
2025-02-16 12:00:00+00:00	False	False	False	False	96519.687083
2025-02-16 16:00:00+00:00	False	False	False	False	96586.259099
2025-02-16 20:00:00+00:00	False	False	True	False	NaN
2025-02-17 00:00:00+00:00	False	False	True	False	NaN
2025-02-17 04:00:00+00:00	False	False	True	False	NaN
2025-02-17 08:00:00+00:00	False	False	True	False	NaN
2025-02-17 12:00:00+00:00	False	False	True	False	NaN
2025-02-17 16:00:00+00:00	False	False	True	False	NaN
2025-02-17 20:00:00+00:00	False	False	True	False	NaN
2025-02-18 00:00:00+00:00	False	False	True	False	NaN
2025-02-18 04:00:00+00:00	False	False	True	False	NaN
2025-02-18 08:00:00+00:00	False	False	True	False	NaN
2025-02-18 12:00:00+00:00	False	False	True	False	NaN
2025-02-18 16:00:00+00:00	False	False	True	False	NaN
2025-02-18 20:00:00+00:00	False	False	True	False	NaN

2025-02-19 00:00:00+00:00	False	False	True	False	NaN
2025-02-19 04:00:00+00:00	False	False	True	False	NaN
2025-02-19 08:00:00+00:00	False	False	False	False	94875.096240
2025-02-19 12:00:00+00:00	False	False	False	False	94840.240035
2025-02-19 16:00:00+00:00	False	False	False	False	94761.145323
2025-02-19 20:00:00+00:00	False	False	False	False	94521.307934

Chandelier Exit:

	ce_l	ce_sh	ce_exit
2025-02-10 00:00:00+00:00	NaN	98401.233386	True
2025-02-10 04:00:00+00:00	NaN	98339.946068	True
2025-02-10 08:00:00+00:00	NaN	98200.556505	True
2025-02-10 12:00:00+00:00	NaN	98258.496108	True
2025-02-10 16:00:00+00:00	NaN	98163.860783	True
2025-02-10 20:00:00+00:00	NaN	97987.592267	True
2025-02-11 00:00:00+00:00	NaN	97903.221838	True
2025-02-11 04:00:00+00:00	95450.588651	NaN	False
2025-02-11 08:00:00+00:00	95477.928859	NaN	False
2025-02-11 12:00:00+00:00	95368.990860	NaN	False
2025-02-11 16:00:00+00:00	NaN	98074.941523	True
2025-02-11 20:00:00+00:00	NaN	98170.653289	True
2025-02-12 00:00:00+00:00	NaN	98148.655733	True
2025-02-12 04:00:00+00:00	NaN	98096.901863	True
2025-02-12 08:00:00+00:00	NaN	97970.350837	True
2025-02-12 12:00:00+00:00	NaN	97619.324662	True
2025-02-12 16:00:00+00:00	NaN	97893.065422	True
2025-02-12 20:00:00+00:00	NaN	97876.330839	True
2025-02-13 00:00:00+00:00	NaN	97752.471828	True
2025-02-13 04:00:00+00:00	NaN	97867.023505	True
2025-02-13 08:00:00+00:00	NaN	97756.021826	True
2025-02-13 12:00:00+00:00	NaN	97720.191026	True
2025-02-13 16:00:00+00:00	NaN	97697.612091	True
2025-02-13 20:00:00+00:00	NaN	97666.875848	True
2025-02-14 00:00:00+00:00	NaN	97571.305475	True
2025-02-14 04:00:00+00:00	NaN	97448.855084	True
2025-02-14 08:00:00+00:00	NaN	97293.579721	True
2025-02-14 12:00:00+00:00	NaN	97315.123692	True
2025-02-14 16:00:00+00:00	95352.442621	NaN	False
2025-02-14 20:00:00+00:00	95286.196720	NaN	False
2025-02-15 00:00:00+00:00	95405.603427	NaN	False
2025-02-15 04:00:00+00:00	95560.752847	NaN	False
2025-02-15 08:00:00+00:00	95706.320166	NaN	False
2025-02-15 12:00:00+00:00	95843.868726	NaN	False
2025-02-15 16:00:00+00:00	95982.971295	NaN	False
2025-02-15 20:00:00+00:00	96130.780823	NaN	False
2025-02-16 00:00:00+00:00	96275.296478	NaN	False
2025-02-16 04:00:00+00:00	96368.003538	NaN	False
2025-02-16 08:00:00+00:00	96486.231522	NaN	False
2025-02-16 12:00:00+00:00	96519.687083	NaN	False
2025-02-16 16:00:00+00:00	96586.259099	NaN	False
2025-02-16 20:00:00+00:00	NaN	97586.476484	True
2025-02-17 00:00:00+00:00	NaN	97524.671244	True
2025-02-17 04:00:00+00:00	NaN	97488.409570	True
2025-02-17 08:00:00+00:00	NaN	97792.155985	True
2025-02-17 12:00:00+00:00	NaN	97905.845178	True
2025-02-17 16:00:00+00:00	NaN	97511.487933	True

2025-02-17 20:00:00+00:00	NaN	97532.882210	True
2025-02-18 00:00:00+00:00	NaN	97501.534039	True
2025-02-18 04:00:00+00:00	NaN	97538.960737	True
2025-02-18 08:00:00+00:00	NaN	97424.961867	True
2025-02-18 12:00:00+00:00	NaN	97576.471846	True
2025-02-18 16:00:00+00:00	NaN	96139.753433	True
2025-02-18 20:00:00+00:00	NaN	96343.299504	True
2025-02-19 00:00:00+00:00	NaN	96311.078334	True
2025-02-19 04:00:00+00:00	NaN	96253.537583	True
2025-02-19 08:00:00+00:00	94875.096240	NaN	False
2025-02-19 12:00:00+00:00	94840.240035	NaN	False
2025-02-19 16:00:00+00:00	94761.145323	NaN	False
2025-02-19 20:00:00+00:00	94521.307934	NaN	False

Exit Prices:

2025-01-07 08:00:00+00:00	99754.730046
2025-01-13 04:00:00+00:00	92383.213798
2025-01-19 16:00:00+00:00	101615.824508
2025-01-22 20:00:00+00:00	102885.736710
2025-02-05 16:00:00+00:00	96785.232201
2025-03-03 20:00:00+00:00	87538.628259
2025-03-18 08:00:00+00:00	81616.385247
2025-03-28 00:00:00+00:00	86064.671424
2025-04-02 16:00:00+00:00	83935.509271
2025-05-04 04:00:00+00:00	95669.618667
2025-05-12 12:00:00+00:00	102065.357192
2025-05-15 00:00:00+00:00	102369.913995
2025-05-25 00:00:00+00:00	107802.732110
2025-06-12 04:00:00+00:00	107537.709077

Name: ce_l, dtype: float64

```
In [75]: entry_times = list(filtered_entry[filtered_entry].index)
exit_times = list(filtered_exit[filtered_exit].index)

# 2) Pair entries with the next exit after each entry
pairs = []
j = 0
for et in entry_times:
    while j < len(exit_times) and exit_times[j] <= et:
        j += 1
    if j < len(exit_times):
        pairs.append((et, exit_times[j]))
        j += 1

close = signals['close']
exit_price = (close * signals['ce_l']).shift(-1)
exits_price = close * filtered_exit
entries_price = close * filtered_entry
profit = pd.Series(0.0, index=filtered_entry.index)

for pair in pairs:
    profit.loc[pair[1]] = exits_price.loc[pair[1]] - entries_price.loc[pair[0]]
profit_cum = profit.cumsum()
profit[profit == 0.0] = np.nan
profit_df = pd.concat([entries_price, exits_price, profit, profit_cum], axis=1)
```

```
print(profit_df.tail())
```

	0	1	2	3
2025-06-29 08:00:00+00:00	0.0	0.0	NaN	-2135.789062
2025-06-29 12:00:00+00:00	0.0	0.0	NaN	-2135.789062
2025-06-29 16:00:00+00:00	0.0	0.0	NaN	-2135.789062
2025-06-29 20:00:00+00:00	0.0	0.0	NaN	-2135.789062
2025-06-30 00:00:00+00:00	0.0	0.0	NaN	-2135.789062

```
In [76]: ## Boolean where in position
y1_arr = signals['close'].values
in_position = pd.Series(pd.NA, index=filtered_entry.index, dtype="boolean")
in_position[filtered_entry] = filtered_entry[filtered_entry]
in_position[filtered_exit] = ~filtered_exit[filtered_exit]
in_position.ffmpeg(inplace=True)
y2 = pd.Series(np.nan, index=filtered_entry.index, dtype="float64")
y2[filtered_entry] = signals['close'][filtered_entry]
y2.ffmpeg(inplace=True)
y2.bfill(inplace=True)
y2_arr = y2.values
in_p = in_position.fillna(False)
in_p_arr = in_p.values
mask_gt = (y1_arr > y2_arr) & in_p_arr
mask_le = (~mask_gt) & in_p_arr
# fill_between = dict(y1=y1_arr, y2=y2_arr, where=in_p_arr, facecolor='green', alpha=0.5)

# 2) Build two fill regions
fill_green = dict(
    y1=y1_arr,
    y2=y2_arr,
    where=mask_gt,
    facecolor='green',
    alpha=0.5
)

fill_red = dict(
    y1=y1_arr,
    y2=y2_arr,
    where=mask_le,
    facecolor='red',
    alpha=0.5
)

# 3) Pass as a list to mplfinance.plot
fill_between = [fill_green, fill_red]
```

```
In [92]: # Combine and color differently
vline_dates = list(filtered_entry[filtered_entry].index) + list(filtered_exit[filtered_exit].index)
vline_colors = (['g'] * filtered_entry.sum()) + (['r'] * filtered_exit.sum())
adx_line = pd.Series(adx_threshold, index=df.index)
zero_line = pd.Series(0.0, index=df.index)
addplots = [
    mpf.make_addplot(psar.iloc[:, 0], type='line', color='blue', width=1.0, label='PSAR'),
    mpf.make_addplot(psar.iloc[:, 1], type='line', width=1.0, label='SAR_sell'),
    mpf.make_addplot(signals['ce_1'], type='line', color='magenta', width=1.0, label='CE_1',
                      secondary_y=False),
```

```

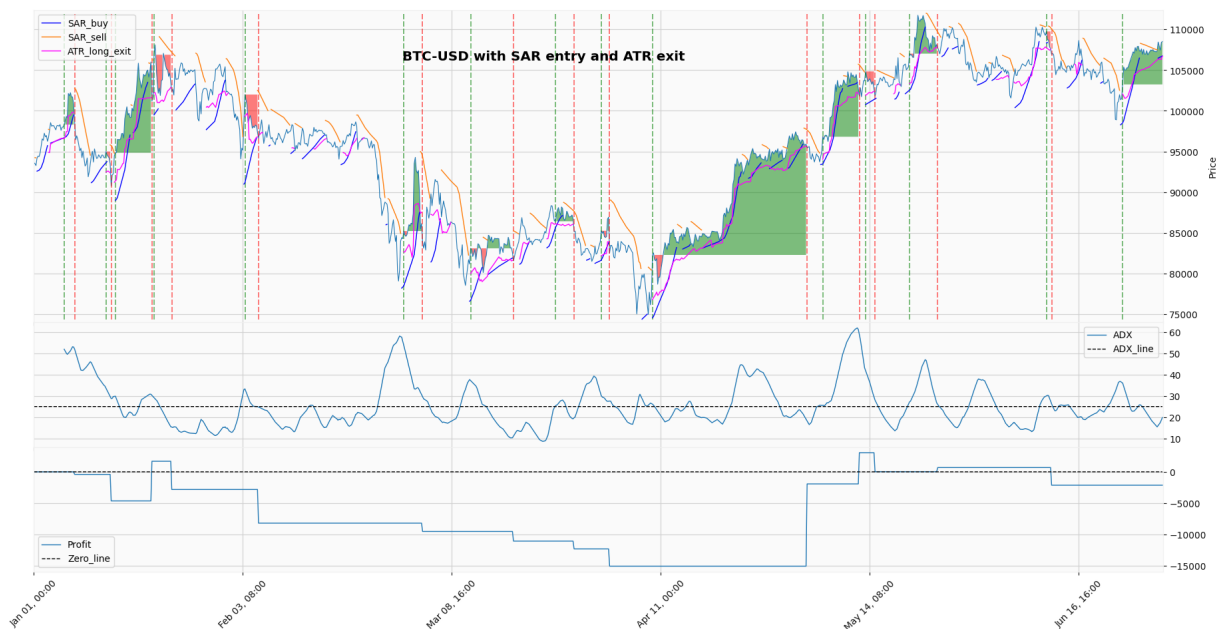
# mpf.make_addplot(signals['ce_sh'], type='line', width=1.0, label='ATR_short_e
#
# secondary_y=False),
mpf.make_addplot(signals['adx'], type='line', width=1.0, label='ADX', panel=1),
mpf.make_addplot(adx_line, type='line', color="black", linestyle='--', width=1.
mpf.make_addplot(profit_cum, type='line', width=1.0, label='Profit', panel=2),
mpf.make_addplot(zero_line, type='line', color="black",linestyle='--', width=1.
# mpf.make_addplot(profit, type='scatter', panel=3),

]

# keep your existing addplots; no need to append invisible lines
all_addplots = addplots

fig, axlist = mpf.plot(
    df,
    type="line",
    volume=False,
    style="yahoo",
    addplot=all_addplots,
    vlines=dict(
        vlines=vline_dates,
        colors=vline_colors,
        linewidths=1.5,
        alpha=0.5,
        linestyle='dashed',
    ),
    figratio=(18, 9),
    figscale=1.7,
    tight_layout=True,
    title=f"{symbols[0]}-USD with SAR entry and ATR exit",
    # xlim=[end - pd.Timedelta(days=150), end],
    returnfig=True,
    fill_between=fill_between,
)

```



```

In [96]: fig = plt.figure(figsize=(16, 4)) # wider figure
ax = plt.gca()

```

```

sns.lineplot(data=profit_df, x=profit_df.index, y=profit_df[3])
sns.lineplot(zero_line,linestyle='--', color="black", linewidth=1.0)
plt.tight_layout()
ax.set_xlim(start, end)
plt.show()

fig2 = plt.figure(figsize=(16, 4)) # wider figure
ax2 = plt.gca()

sns.scatterplot(data=profit_df, x=profit_df.index, y=profit_df[2], hue=profit_df[2],
                palette={True: 'green', False: 'red'})
sns.lineplot(zero_line,linestyle='--', color="black", linewidth=1.0)
plt.tight_layout()
ax2.set_xlim(start, end)
plt.show()

```

